

Laboratorio de Ataque y Defensa

Duración: 5 horas **Qué necesitan tener listo:** la API de Numerología corriendo, con al menos 3 colecciones, sus rutas, controllers y `express-validator`

De qué se trata esto

Uno no aprende a validar datos escribiendo validaciones. Uno aprende cuando ve lo que pasa **cuando no las hay**.

Hoy no van a construir nada nuevo. Hoy le van a tirar piedras a la API de su compañero hasta que algo se rompa, van a anotar exactamente cómo se rompió, y después cada uno va a reparar la suya con el reporte que le entregaron.

el que dice quién tiene la razón es el servidor. Si la API responde con un error 500 y un montón de texto rojo de Mongoose, está mal. Si responde con un 400 y un mensaje claro, está bien. No hay nada que discutir ni a quién preguntarle.

Las reglas

1. Trabajan en parejas, pero **cada uno con su propio repositorio**. Nadie toca el código del otro.
 2. Al compañero solo se le mandan peticiones HTTP. Nada más.
 3. Si encuentras una falla, tienes que poder demostrarla: qué método usaste, a qué URL, qué mandaste en el body, y qué te respondió. Una falla que no puedes reproducir no existe.
 4. **No le digas a tu compañero cómo arreglar las cosas.** Le dices qué se rompió y ya. El diagnóstico es tarea de él. Si le das la respuesta, le estás quitando la parte que sirve.
 5. Van anotando sobre la marcha. Al final nadie se acuerda de nada.
-

Cómo le das acceso a tu API al compañero

Tu API corre en `localhost:3000`, y ese `localhost` es tu computador. Tu compañero no lo puede ver desde el suyo.

La solución más fácil está dentro de VS Code y se llama **Port Forwarding**. Con eso VS Code te genera una URL pública temporal que apunta a tu servidor local. Tu compañero abre esa URL desde su Postman y le llega directo a tu máquina.

Paso a paso

1. Inicia sesión en VS Code

Abajo a la izquierda hay un ícono de persona (Accounts). Haz clic e inicia sesión con tu cuenta de GitHub o de Microsoft. Sin esto el port forwarding no funciona.

2. Levanta tu servidor

```
npm run dev
```

Confirma que dice que está corriendo en el puerto 3000. Si no está corriendo, no hay nada que exponer.

3. Abre el panel de puertos

En el panel de abajo, donde está la Terminal, hay una pestaña que dice **PORTS** (al lado de Terminal, Output, Debug Console).

Si no la ves: **Ctrl + Shift + P** → escribe **Ports: Focus on Ports View** → Enter.

4. Reenvía el puerto

Clic en **Forward a Port** (o en el botón **+**). Escribe **3000** y dale Enter.

VS Code te va a generar una dirección parecida a esta:

```
https://a1b2c3d4-3000.use.devtnnls.ms
```

5. Ponlo en público (este paso es el que todos olvidan)

Por defecto el puerto queda en **Private**, y si se lo mandas así a tu compañero le va a salir una pantalla pidiéndole que inicie sesión. Va a creer que tu API está caída y no lo está.

Clic derecho sobre el puerto en la lista → **Port Visibility** → **Public**.

En la columna "Visibility" debe decir **Public**. Ahí sí.

6. Copia la URL y pásasela a tu compañero

Clic derecho → **Copy Local Address**.

Tu compañero la usa igual que usaría localhost, solo cambiando la base:

Tú en tu máquina: `http://localhost:3000/api/v1/usuarios`

Tu compañero desde acá: `https://a1b2c3d4-3000.use.devtnnls.ms/api/v1/usuarios`

Cosas que van a pasar

- **La primera vez sale una pantalla de advertencia** de Dev Tunnels antes de dejar pasar. Hay un botón para continuar. Es normal, pasa una sola vez por navegador.
- **Si cierran VS Code o apagan el servidor, el túnel se cae**. Cuando lo vuelvas a abrir, la URL puede ser distinta. Avísale a tu compañero.
- **"Public" significa público de verdad**. Cualquiera con ese link entra a tu API. Es un ejercicio de clase con datos de prueba, no hay drama, pero **cuando terminen hoy, cierren el puerto** (clic derecho → Stop Forwarding Port). Acostúmbrense a cerrar lo que abren.
- **Si Postman te da timeout**, revisa lo obvio primero, en este orden: ¿el servidor está corriendo?, ¿el puerto dice Public?, ¿copiaste bien la URL?

Si el port forwarding no les sirve

Plan B: trabajen en el mismo computador, turnándose. El que ataca se sienta con su Postman en la máquina del que defiende y usa `localhost` normal. Es más incómodo pero funciona igual, y el ejercicio no pierde nada. Aunque sería bueno que aprendan algo nuevo con lo del puerto.

Bloque 0 — Preparar

Cada uno por su lado:

1. Levanta el servidor y confirma que responde.
2. Mete datos de prueba: **mínimo 3 documentos por colección**, y al menos 2 relaciones `ref` que apunten a algo real. Una base de datos vacía no se puede atacar, no hay de dónde agarrar.
3. Expón el puerto siguiendo el tutorial de arriba.
4. Mándale a tu compañero: la URL del túnel y la lista de endpoints que tienes.

Si tu API ni siquiera arranca, ese es tu primer hallazgo. Anótalo, arréglalo, y sigue.
Pero el tiempo sale del tuyo, no del bloque de ataque.

Bloque 1 — A romper cosas

Agarras la lista de endpoints de tu compañero y le pasas los **14 ataques** de abajo, uno por uno, anotando todo.

Formato para cada ataque:

ATAQUE #__ : (nombre)

Petición: MÉTODO /ruta

Body: { ... }

Respondió: código HTTP + lo que devolvió

Veredicto: DEFENDIDO / VULNERABLE

Qué noté: (qué esperabas que pasara vs qué pasó)

Los 14 ataques

Primero, lo básico: datos malos

1. **Falta lo obligatorio.** POST con el body casi vacío, sin los campos requeridos. ¿Te responde un 400 diciéndote qué falta, o revienta con un 500 de Mongoose?
2. **Body totalmente vacío.** POST con `{}`. Mismo criterio.
3. **Tipos cambiados.** Manda un número donde va texto. Un texto donde va una fecha. Un texto donde va un booleano.
4. **Vacío disfrazado.** Manda " " y después " " (solo espacios) en un campo obligatorio. El segundo es el interesante: ¿pusieron `.trim()` antes del `.notEmpty()` o no?
5. **Valor inventado en un enum.** Si hay un campo con lista cerrada de opciones, mándale una que no exista.

6. **Texto gigante.** Manda un string de 10.000 caracteres en un campo de texto. ¿Hay tope, o se traga lo que sea?

Ahora lo que no debería poder tocar el cliente

7. **Mass assignment.** Busca en el schema de tu compañero un campo que el usuario **no debería** poder decidir: un **activo**, un contador de algo, una bandera de administrador, lo que sea. Mándalo en el POST de creación. Después haz un GET y mira si se guardó. Si se guardó, ahí tienes.
8. **Lo mismo pero por la ventana.** Repite el ataque anterior con **PUT** en vez de **POST**. Es muy común que protejan la creación y se olviden de la actualización.

Los ids y las rutas

9. **Id que no es un id.** **GET /recurso/123abc**. ¿Un 400 decente, o un 500 con un **CastError** de Mongoose colgando por ahí?
10. **Id válido pero que no existe.** Toma un id real y cámbiale el último carácter por otro que también sea válido. ¿Te da un 404 claro, un 200 con **null**, o se cae?
11. **Método que no existe.** Hazle un **DELETE** a una ruta que solo definió como **GET**. Mira qué contesta.

Y aquí viene lo bueno: las relaciones

12. **Referencia a la nada.** Crea un documento cuyo campo **ref** apunte a un ObjectId con formato correcto pero que **no existe** en la otra colección. ¿Lo dejó guardar? Ahora haz el GET con **populate** sobre ese documento. Mira qué te devuelve el campo poblado.
13. **Borrar algo del que otros dependen.** Elimina un documento que esté referenciado por otro (borra el artista que tiene canciones). Después consulta la canción con **populate**. ¿Qué quedó de esa relación?
14. **Actualizar solo un campo.** Haz un **PUT** mandando **un solo campo** de un documento que tiene cinco. Después haz un GET de ese documento y míralo.

No adivines el resultado, compruébalo. Si los otros cuatro campos siguen ahí, averigua *por qué* (¿qué hace Mongoose con los valores **undefined** en un update?). Si desaparecieron, encontraste algo grave. En los dos casos, la explicación va en el reporte.

Lo que entregan de este bloque

informe-ataque.md — los 14 ataques con su evidencia y su veredicto, más el conteo final: **X defendidos / Y vulnerables**.

Bloque 2 — Intercambio

1. Se pasan los reportes. **Sin conversar sobre soluciones**. Se lee y punto.
 2. Cada uno agrupa sus propias fallas en tres montones:
 - **CRÍTICO** — deja guardar datos corruptos o inconsistentes en la base de datos
 - **GRAVE** — responde con 500, o le muestra al cliente cosas internas que no debería (stack traces, errores crudos de Mongoose)
 - **MENOR** — funciona, pero el mensaje es malo o el código HTTP no es el que corresponde
 3. Escribe en tu bitácora, **antes de tocar una sola línea de código**, en qué orden vas a arreglar las cosas. Esa lista es un compromiso: al final la vas a comparar con lo que realmente hiciste.
-

Bloque 3 — A reparar

Arreglas **tu** API, en el orden que definiste.

Por cada arreglo, anota:

REPARACIÓN del ATAQUE #__

Por qué falló: la causa real, no el síntoma

Dónde lo arreglé: model / validator / middleware / controller / route

Qué cambié: el código, antes y después

Cómo lo comprobé: la misma petición del ataque, con la respuesta nueva

Una regla que sí es rígida: cada falla se arregla en **una sola capa**. Si te encuentras poniendo el mismo `if` en tres controllers, estás arreglando en el lugar equivocado — eso se sube a un validator o a un middleware. Y si terminas con la misma validación en el validator y en el controller, algo quedó mal ubicado.

Sobre los ataques 12 y 13: les adelanto algo, porque si no se van a estrellar. Ninguno de esos dos lo resuelve `express-validator` solo, ni el schema de Mongoose solo. Van a tener que decidir **dónde** vive esa responsabilidad. No hay una única respuesta correcta — pero sí hay respuestas mal argumentadas, y esas se notan.

Bloque 4 — Segunda ronda

Vuelves a correr **los mismos 14 ataques** contra la API ya reparada de tu compañero.

Anota tres cosas:

- Cuántos de los hallazgos originales quedaron realmente cerrados
 - **Regresiones:** algo que antes funcionaba bien y ahora no. Esto pasa más de lo que creen, y hay que salir a buscarlo — corre también un CRUD normal completo, no solo los ataques.
 - Fallas nuevas que aparecieron durante la reparación
-

Bloque 5 — Defensa escrita

Esto es individual. Va en `defensa-tecnica.md`.

Cada respuesta tiene que citar tu propio código, pegando el fragmento y diciendo en qué archivo está. Respuestas generales o sacadas de internet no valen: la pregunta es sobre *tu* implementación, no sobre el tema en abstracto.

1. Pega una regla de tu validator y dime **cuál de los 14 ataques** bloquea. Si no bloquea ninguno, ¿para qué la escribiste?
2. Hay datos inválidos que rechaza `express-validator` y **también** rechazaría el schema de Mongoose. Busca un caso concreto en tu API donde pase eso. ¿Es repetir por repetir, o es defensa en capas? Respóndeme con lo que viste hoy, no con teoría.

3. Del ataque 12: cuéntame qué hizo tu API y qué decidiste hacer al respecto. Si decidiste **no** arreglarlo, explícame por qué esa es una decisión razonable y no simplemente que se te quedó.
 4. Del ataque 14: qué pasó realmente y **por qué**. Esta no la puedes responder si no lo comprobaste.
 5. Pega el pedazo de código donde impides el mass assignment. Explícame por qué el `strict: true` de Mongoose no bastaba por sí solo.
 6. ¿Cuál falla te costó más entender, y qué fue lo que te confundió al principio?
-

Lo que entregan

Todo en una carpeta `laboratorio/` dentro de su repositorio:

Archivo	Qué lleva
<code>informe-ataque.md</code>	Los 14 ataques al compañero, ronda 1 y ronda 2
<code>bitacora-reparacion.md</code>	La priorización, y cada arreglo con su diagnóstico y comprobación
<code>defensa-tecnica.md</code>	Las 6 preguntas, respondidas con código propio
Los commits	Con mensajes que digan algo

Sobre los commits: `fix(validacion): rechaza ObjectId invalido en GET /:id`.
No `arreglos`, no `cambios`, no `ya sirve`.

Cómo se van a calificar ustedes mismos

Antes de entregar, cada uno se hace su propia autoevaluacion. Después la contrasta con la evidencia.

Qué se mira	Puntos
Los 14 ataques hechos y documentados de forma reproducible	20
La clasificación por severidad hecha antes de tocar código	10
Cada reparación con el diagnóstico de la causa (no del síntoma)	20
Arreglos en la capa correcta, sin lógica duplicada	15
Segunda ronda hecha, buscando regresiones a propósito	10
Defensa escrita citando código propio en cada respuesta	20
Commits que se entienden	5

Las preguntas 3 y 4 de la defensa pesan más que las otras. Son las únicas que no se pueden responder sin haber ejecutado y observado. Si están llenas de teoría genérica, se nota apenas se leen.

Si te quedas atascado

Hoy no estoy. Así que el procedimiento es este, en orden:

1. **Lee el error completo.** No la primera línea — todo. La mayoría de las veces Mongoose y Express te están diciendo textualmente qué pasó, y uno no lo lee.
2. **Ubica dónde se cae.** ¿Es la ruta, el validator o el controller? Mete un `console.log` al principio de cada uno y mira hasta dónde llega la petición. Si nunca entra al controller, el problema está antes.
3. **Simplifica el caso.** Quita campos del body hasta que funcione, después vuélvelos a meter de a uno. El que lo rompe es el culpable.

4. **Documentación oficial**, en este orden: `express-validator`, después Mongoose, después Express. Los tres tienen buena documentación con ejemplos. Antes de irte a un foro, revisa ahí.
 5. Si llevas 20 minutos en lo mismo: **anótalo como pendiente abierto** en la bitácora, con lo que intentaste, y sigue con lo siguiente. Un problema documentado sin resolver vale mucho más que dos horas estancado sin dejar rastro.
-

Casos especiales

Si son número impar: hacen un trío en cadena. A ataca a B, B ataca a C, C ataca a A. Funciona exactamente igual.

Si les sobra tiempo: inventen un ataque #15 que no esté en la lista y documéntenlo igual. Los mejores me los quedo para el próximo grupo.